Tilting the Tree Back -- Balance Factor and the Four Rotations

Lecture 12

Data Structures and Algorithms

Part 1: The Need for Balance

When BSTs Go Wrong -- A Recap

Last lecture, we saw that inserting **sorted data** into a BST produces a degenerate tree:

```
Insert 10, 20, 30, 40, 50:                    Insert 50, 30, 70, 20, 40:

  10                                                50
    \                                              /  \
    20          Height = 4                       30    70      Height = 2
      \         Search = O(n)                   / \             Search = O(log n)
      30                                       20  40
        \
        40
          \
          50
```

Same 5 values. One tree gives O(n). The other gives O(log n).

The root cause: nothing prevents the tree from becoming lopsided. We need a mechanism that **automatically rebalances** the tree after every insertion and deletion.

What Does "Balanced" Mean?

A tree is **balanced** if its height is kept at O(log n). Several strategies exist:

Strategy	Key Idea	Height Guarantee
AVL Tree	Height difference between subtrees <= 1	<= 1.44 log n
Red-Black Tree	Color-based rules limit imbalance	<= 2 log n
Splay Tree	Move recently accessed nodes to root	Amortized O(log n)

All of these use a fundamental operation called **rotation** to restructure the tree without violating the BST property.

Today's focus: AVL trees -- the oldest and most intuitive self-balancing BST.

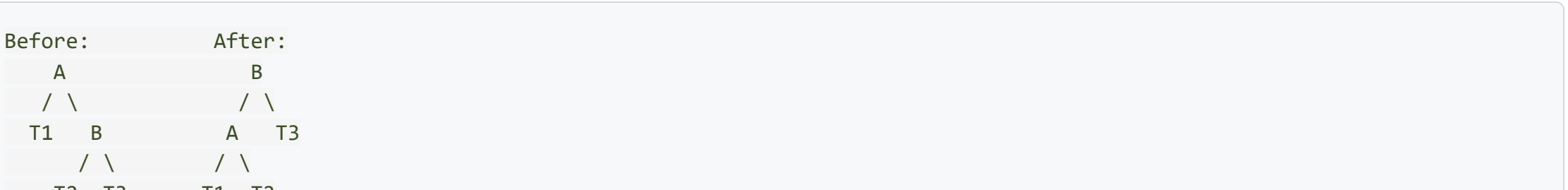
The Key Operation: Rotation

A **rotation** is a local restructuring that changes the **shape** of the tree but preserves the **BST ordering**.

Right Rotation at node A:

"Pull B up, A goes down to the right. B's right child (T2) becomes A's left child."

Left Rotation at node A:



"Pull B up, A goes down to the left. B's left child (T2) becomes A's right child."

Why Do Rotations Preserve BST Order?

Consider the right rotation at A:

```
Before:  T1 < B < T2 < A < T3     (in-order: T1, B, T2, A, T3)

After:   T1 < B < T2 < A < T3     (in-order: T1, B, T2, A, T3)
```

The in-order sequence does not change! Rotations only change which node is "on top" -- they never change the relative ordering of elements.

This is the powerful guarantee that makes rotations safe: we can reshape the tree as aggressively as we want, and the BST property is never violated.

Rotation Cost

Each rotation involves **a constant number of pointer changes** (3 pointers reassigned). So a single rotation takes **O(1)** time.

Part 2: AVL Trees -- Definition and Balance Factor

What is an AVL Tree?

Named after its inventors **Adelson-Velsky and Landis** (1962) -- the first self-balancing BST ever proposed.

AVL Property: For **every** node in the tree, the height difference between its left and right subtrees is at most 1.

Balance Factor (BF)

$$BF(node) = height(left\ subtree) - height(right\ subtree)$$

Allowed values: {-1, 0, +1}

- BF = +1: left subtree is 1 taller (slightly left-heavy)
- BF = 0: both subtrees have equal height (perfectly balanced)
- BF = -1: right subtree is 1 taller (slightly right-heavy)

If any node has BF = +2 or BF = -2 after an insertion or deletion, the tree is **unbalanced** and we must **rotate** to fix it.

AVL Tree: Example with Balance Factors

```
                  50 [0]
                /      \
          30 [+1]      70 [-1]
          /   \            \
      20 [0]  40 [0]      80 [0]
```

This is a valid AVL tree. Every node has BF in {-1, 0, +1}.

Now, insert **10**:

```
                  50 [+1]                50 [+2]     <-- VIOLATION!
                /      \              /      \
          30 [+1]      70        30 [+2]    70     <-- VIOLATION!
          /   \                  /   \
      20 [0]  40             20 [+1]  40
                            /
                          10 [0]
```

Node **30** now has BF = +2 and node **50** has BF = +2. The tree is unbalanced! We need to fix the **first unbalanced ancestor** -- which is node **30** (closest to the insertion point).

Height Guarantee

An AVL tree with n nodes has a height of at most **1.44 x log2(n)**. This means:

Nodes (n)	Max Height (BST)	Max Height (AVL)
7	6	3
15	14	5
100	99	9
1,000	999	14
1,000,000	999,999	28

AVL trees guarantee O(log n) operations regardless of the insertion order. Even inserting fully sorted data produces a balanced tree.

The price we pay: maintaining balance after every insert and delete, which involves identifying the imbalance case and performing the right rotation.

Part 3: The Four Imbalance Cases

How to Identify the Imbalance Case

When a node becomes unbalanced (BF = +2 or -2), we identify the case by looking at **where the new node was inserted** relative to the unbalanced node:

Case	Unbalanced Node BF	Child's BF	Inserted Where?
LL	+2 (left-heavy)	+1 (left-heavy)	Left subtree of left child
RR	-2 (right-heavy)	-1 (right-heavy)	Right subtree of right child
LR	+2 (left-heavy)	-1 (right-heavy)	Right subtree of left child
RL	-2 (right-heavy)	+1 (left-heavy)	Left subtree of right child

Mnemonic: The case name tells you the **path** from the unbalanced node to the inserted node:

- LL = went Left, then Left again
- LR = went Left, then Right
- The rotation direction is the **opposite** of the imbalance

LL Case -- Single Right Rotation

Scenario: Inserted in the **left subtree of the left child**. The tree is top-heavy to the left.

```
Before (BF at A = +2):                After right rotation at A:

        A [+2]                                  B [0]
       / \                                     / \
    B [+1] T3                               T1   A [0]
   / \                                          / \
  T1  T2                                      T2  T3
  ^
  new node inserted here
```

One right rotation at A fixes the imbalance. B becomes the new root of this subtree.

Concrete Example:

```
Insert 10, 20, 30:          Insert 5:                After Right Rotation at 20:

    30                            30                          30
   /                             /                           /
  20           -->              20 [+2]      -->             10
 /                             /                           /  \
10                          10 [+1]                       5   20
                           /
                          5

LL case at node 20: Right rotate at 20
```

RR Case -- Single Left Rotation

Scenario: Inserted in the **right subtree of the right child**. Mirror of LL.

```
Before (BF at A = -2):                After left rotation at A:

    A [-2]                                    B [0]
   / \                                       / \
  T1  B [-1]                               A [0] T3
     / \                                   / \
    T2  T3                               T1  T2
          ^
    new node inserted here
```

One left rotation at A fixes the imbalance.

Concrete Example:

```
Insert 10, 20, 30:

  10                     10 [-2]                    20
    \                      \           -->         /  \
    20        -->          20 [-1]                10    30
                             \
                             30

RR case at node 10: Left rotate at 10
```

LR Case -- Left-Right Double Rotation

Scenario: Inserted in the **right subtree of the left child**. This creates a "zigzag" that a single rotation cannot fix.

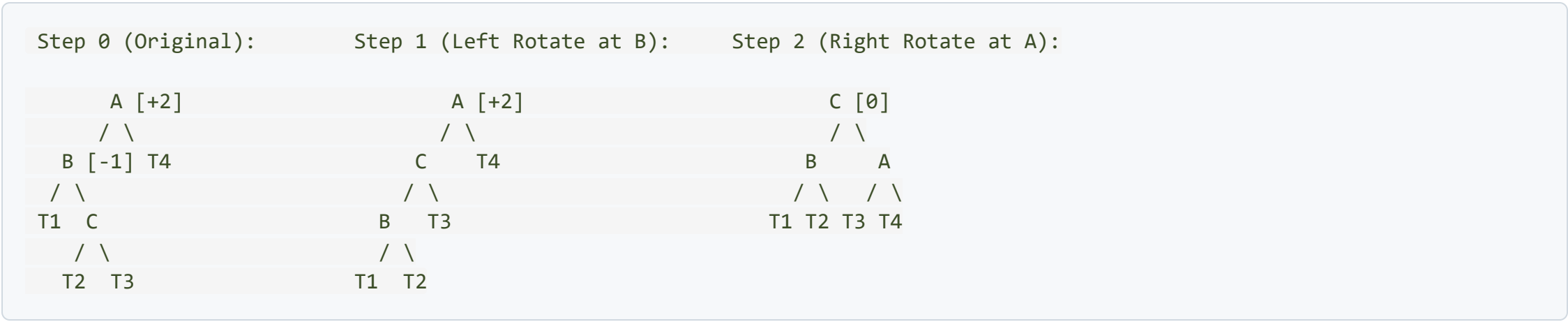
```
Before (BF at A = +2):

        A [+2]
       / \
    B [-1] T4
   / \
  T1  C
     / \
    T2  T3
    ^     ^
   (inserted in T2 or T3)
```

Why a single right rotation at A fails: It would place the imbalanced subtree C in an awkward position, not fixing the height issue.

Solution: Two rotations.

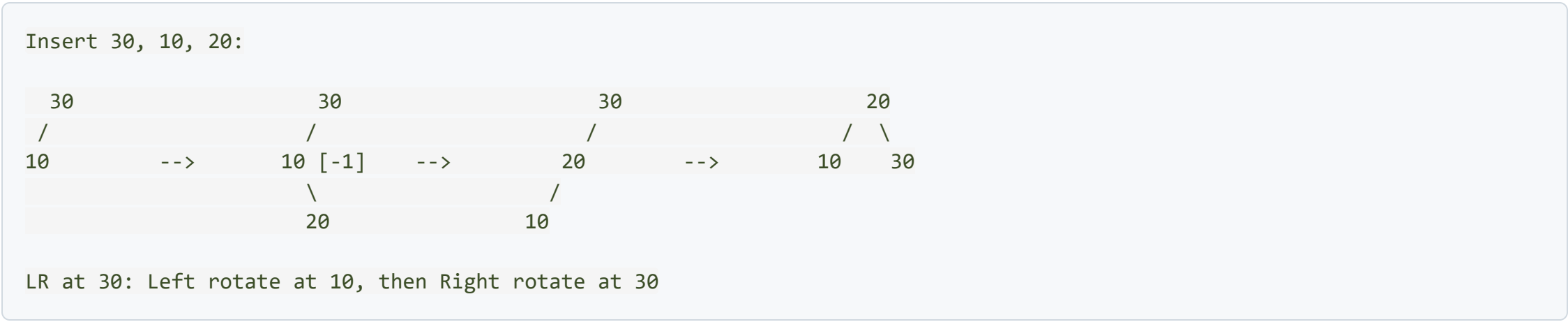
1. **Left rotate at B** (converts the zigzag to a straight line -- now it looks like LL)
2. **Right rotate at A** (fixes the imbalance, just like the LL case)

LR Case -- Step by Step



After both rotations: C becomes the new root of this subtree, B and A become its children, and all subtrees T1-T4 land in their correct positions.

Concrete Example:



RL Case -- Right-Left Double Rotation

Scenario: Inserted in the **left subtree of the right child**. Mirror of LR.

```
Step 0 (Original):          Step 1 (Right Rotate at B):     Step 2 (Left Rotate at A):

    A [-2]                        A [-2]                          C [0]
   / \                           / \                             / \
  T1  B [+1]                   T1   C                          A     B
     / \                           / \                        / \   / \
    C   T4                       T2   B                     T1 T2 T3 T4
   / \                               / \
  T2  T3                           T3  T4
```

1. **Right rotate at B** (converts zigzag to straight line -- now looks like RR)
2. **Left rotate at A** (fixes imbalance)

Concrete Example:

```
Insert 10, 30, 20:

10                    10                      10                    20
  \                     \                       \                  /  \
  30       -->          30 [+1]   -->             20      -->      10    30
                        /                          \
                      20                            30

RL at 10: Right rotate at 30, then Left rotate at 10
```

The Four Cases -- Summary Table

Case	Condition	Fix	Rotations
LL	BF = +2, left child BF = +1	Single Right Rotation	1
RR	BF = -2, right child BF = -1	Single Left Rotation	1
LR	BF = +2, left child BF = -1	Left then Right Rotation	2
RL	BF = -2, right child BF = +1	Right then Left Rotation	2

Pattern to remember:

- **Single rotation** cases (LL, RR): the path from the unbalanced node to the inserted node is a **straight line**
- **Double rotation** cases (LR, RL): the path makes a **zigzag**
- For a single rotation: rotate in the **opposite** direction of imbalance
- For a double rotation: first straighten the zigzag, then rotate

Part 4: AVL Insertion -- Complete Algorithm

AVL Insertion: Step-by-Step Algorithm

1. **Insert** the new value as in a regular BST (it becomes a leaf)
2. **Walk back up** from the new node to the root, updating balance factors along the way
3. At each ancestor, check: is $|BF| > 1$?
    - **No:** Continue walking up
    - **Yes:** Identify the case (LL, RR, LR, RL). Apply the corresponding rotation. **Stop** -- only one rotation (single or double) is needed per insertion
4. Update the heights of affected nodes after rotation

Key Property of AVL Insertion:

At most one rotation (which may be a double rotation = two single rotations) is ever needed to restore balance after an insertion. This is because fixing the nearest unbalanced ancestor also fixes any imbalance that propagated upward.

Complexity: O(log n) for the BST insert + O(1) for the rotation = **O(log n)** total.

Complete Worked Example: Insert 30, 20, 10

```
Step 1: Insert 30                    Step 2: Insert 20

  30 [0]                               30 [+1]
                                      /
                                    20 [0]
```

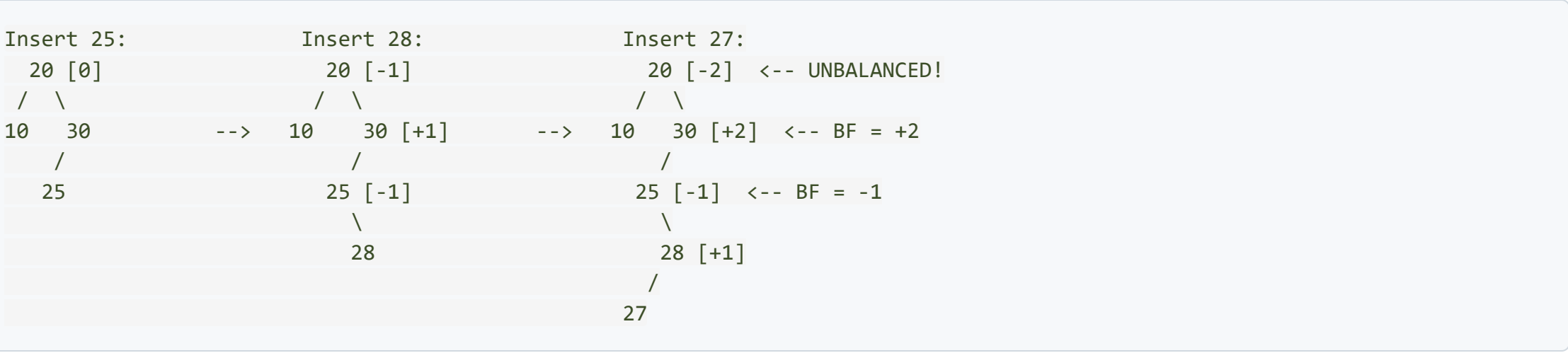
No imbalance yet. Balance factors are within {-1, 0, +1}.

```
Step 3: Insert 10

    30 [+2]      <-- BF = +2, UNBALANCED!
   /
  20 [+1]        <-- Left child BF = +1
 /
10 [0]

Case: LL (went left from 30, then left from 20)
Fix: Single RIGHT rotation at 30
```

```
After rotation:

  20 [0]
 /  \
10   30       All BFs are 0 -- tree is balanced!
```

Complete Worked Example: Insert 30, 20, 10, 25, 28, 27

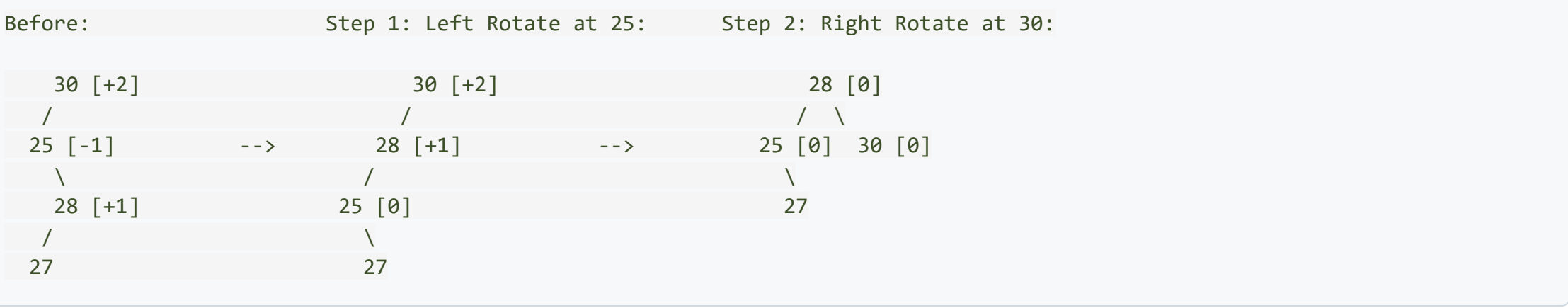
Continuing from the balanced tree after inserting 30, 20, 10:



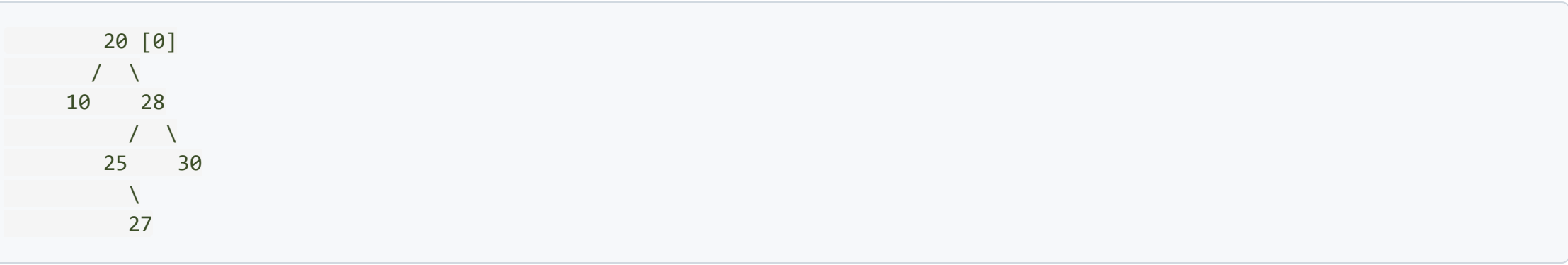
At node 30: BF = +2 (left-heavy), left child 25 has BF = -1 (right-heavy).

Case: LR (zigzag: left from 30, then right from 25)

Worked Example: LR Rotation at 30



After rotation:



All balance factors are restored. The tree is balanced!

AVL Insertion: Rotation Code

```
// Right rotation
struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;          // y goes down to the right
    y->left = T2;          // T2 becomes y's left child

    // Update heights
    y->height = 1 + max(getHeight(y->left), getHeight(y->right));
    x->height = 1 + max(getHeight(x->left), getHeight(x->right));

    return x;              // x is the new root of this subtree
}

// Left rotation
struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;           // x goes down to the left
    x->right = T2;         // T2 becomes x's right child

    x->height = 1 + max(getHeight(x->left), getHeight(x->right));
    y->height = 1 + max(getHeight(y->left), getHeight(y->right));

    return y;
}
```

AVL Insertion: Complete Insert Function

```
struct Node* insertAVL(struct Node* node, int key) {
    // 1. Standard BST insertion
    if (node == NULL) return createNode(key);
    if (key < node->data)      node->left  = insertAVL(node->left, key);
    else if (key > node->data) node->right = insertAVL(node->right, key);
    else return node;  // Duplicate -- do nothing

    // 2. Update height of this ancestor node
    node->height = 1 + max(getHeight(node->left), getHeight(node->right));

    // 3. Get balance factor
    int balance = getBalance(node);

    // 4. Check for imbalance and apply rotations
    // LL Case
    if (balance > 1 && key < node->left->data)
        return rightRotate(node);
    // RR Case
    if (balance < -1 && key > node->right->data)
        return leftRotate(node);
    // LR Case
    if (balance > 1 && key > node->left->data) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    // RL Case
    if (balance < -1 && key < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;  // No imbalance
}
```

Understanding the Insert Code

The Helper Functions

```
int getHeight(struct Node* node) {
    if (node == NULL) return -1;       // Empty tree has height -1
    return node->height;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int getBalance(struct Node* node) {
    if (node == NULL) return 0;
    return getHeight(node->left) - getHeight(node->right);
}

struct Node* createNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data   = key;
    node->left   = NULL;
    node->right  = NULL;
    node->height = 0;    // New node is a leaf with height 0
    return node;
}
```

The Node struct now includes a `height` field:

```
struct Node {
    int data;
```

Why the Code Identifies Cases by Comparing Keys

The typical way to identify cases is by checking the child's balance factor. But there is a shortcut when inserting a **known key**:

```
// LL: balance > 1  AND  key < node->left->data
//     (key went into the LEFT subtree of the LEFT child)

// LR: balance > 1  AND  key > node->left->data
//     (key went into the RIGHT subtree of the LEFT child)
```

This works because during insertion, we know exactly which key was inserted. If the key is less than the left child's data, it must have gone into the left child's left subtree (LL). If greater, it went into the left child's right subtree (LR).

Alternative approach (used in deletion, where we do not have a single key to compare):

```
if (balance > 1 && getBalance(node->left) >= 0)  // LL
if (balance > 1 && getBalance(node->left) < 0)   // LR
if (balance < -1 && getBalance(node->right) <= 0) // RR
if (balance < -1 && getBalance(node->right) > 0)  // RL
```

Part 5: AVL Tree Deletion

Why Deletion is Different from Insertion

Recall: AVL insertion requires at most **one rotation** to restore balance. Deletion is more complex.

The key difference:

- After **insertion**, fixing the nearest unbalanced ancestor reduces the subtree height back to what it was *before* insertion -- so no further ancestors can be affected.
- After **deletion**, fixing one unbalanced node may **reduce the subtree height**, which can trigger imbalance at the **parent**, then the **grandparent**, cascading all the way up to the root.

Deletion may require up to O(log n) rotations -- one at each level of the tree.

Deletion Algorithm:

1. Delete the node using standard BST deletion (3 cases: leaf, one child, two children)
2. Walk back up to the root, updating heights and balance factors
3. At each ancestor with $|BF| > 1$, identify the case and apply the appropriate rotation
4. **Keep going** -- do not stop after the first rotation!

AVL Deletion: Identifying the Case

During deletion, we cannot compare against an inserted key (there is none). Instead, we check the **balance factor of the child**:

```
int balance = getBalance(node);

// Left-heavy (BF = +2)
if (balance > 1 && getBalance(node->left) >= 0)   // LL
    return rightRotate(node);
if (balance > 1 && getBalance(node->left) < 0) {   // LR
    node->left = leftRotate(node->left);
    return rightRotate(node);
}

// Right-heavy (BF = -2)
if (balance < -1 && getBalance(node->right) <= 0)  // RR
    return leftRotate(node);
if (balance < -1 && getBalance(node->right) > 0) {  // RL
    node->right = rightRotate(node->right);
    return leftRotate(node);
}
```

The logic is the same as insertion -- only the case identification changes (child's BF instead of key comparison).

AVL Deletion: Worked Example

Delete 5 from this AVL tree:

After removing leaf 5, update balance factors walking up:

```
Node 10: height(left) = -1, height(right) = 0
         BF = -1 - 0 = -1        OK

Node 20: height(left) = 1, height(right) = 2
         BF = 1 - 2 = -1         OK
```

No imbalance -- sometimes deletion does not cause any rotation at all.

AVL Deletion: Cascading Rotation Example

Consider this AVL tree:

```
              50
             /  \
           30     70
          /      / \
        20     60   80
       /
      10
```

Delete 60:

```
              50
             /  \
           30     70 [-2]     <-- BF = -2, right child 80 has BF = 0
          /         \
        20           80
       /
      10
```

At node 70: BF = -2 (right-heavy), right child BF = 0. Treat as **RR case**. Left rotate at 70:

```
              50
             /  \
           30     80
          /      /
        20     70
       /
      10
```

Cascading Rotation Example (continued)

After fixing node 70, update balance factors upward:

```
              50 [+2]      <-- BF = +2, STILL UNBALANCED!
             /  \
           30     80
          /      /
        20     70             Node 50: left subtree height = 3,
       /                              right subtree height = 1
      10                              BF = 3 - 1 = +2
```

At node 50: BF = +2, left child 30 has BF = +1. **LL case**. Right rotate at 50:

```
              30
             /  \
           20     50
          /      /  \
        10     80    (empty)
              /
            70
```

Wait -- that does not look right. Let us be more careful:

```
              30
             /  \
           20     50
          /      / \
        10     70   80
```

Two rotations were needed -- one at level 70, another cascading to level 50. This is why deletion is more expensive

AVL Deletion: Complete Code

```
struct Node* deleteAVL(struct Node* root, int key) {
    // 1. Standard BST deletion
    if (root == NULL) return NULL;
    if (key < root->data)      root->left  = deleteAVL(root->left, key);
    else if (key > root->data) root->right = deleteAVL(root->right, key);
    else {
        if (root->left == NULL || root->right == NULL) {
            struct Node* temp = root->left ? root->left : root->right;
            if (temp == NULL) { temp = root; root = NULL; }
            else *root = *temp;
            free(temp);
        } else {
            struct Node* succ = root->right;
            while (succ->left != NULL) succ = succ->left;
            root->data = succ->data;
            root->right = deleteAVL(root->right, succ->data);
        }
    }
    if (root == NULL) return NULL;

    // 2. Update height
    root->height = 1 + max(getHeight(root->left), getHeight(root->right));

    // 3. Rebalance (same 4 cases as insertion)
    int balance = getBalance(root);
    if (balance > 1 && getBalance(root->left) >= 0) return rightRotate(root);
    if (balance > 1 && getBalance(root->left) < 0)
        { root->left = leftRotate(root->left); return rightRotate(root); }
    if (balance < -1 && getBalance(root->right) <= 0) return leftRotate(root);
    if (balance < -1 && getBalance(root->right) > 0)
        { root->right = rightRotate(root->right); return leftRotate(root); }
    return root;
}
```

AVL Tree: Full Complexity Summary

Operation	Time	Rotations
Search	O(log n)	0
Insert	O(log n)	At most 1 (single or double)
Delete	**O(log n)**	**Up to O(log n) -- can cascade**
Find Min/Max	O(log n)	0

Pros of AVL trees:

- Guaranteed O(log n) for all operations
- Strictly balanced -- height is at most 1.44 x log2(n)
- Best for **search-intensive** applications

Cons of AVL trees:

- Deletion can require multiple rotations (cascading)
- Extra storage: one integer (height) per node
- Slightly complex implementation

This raises a question: Is there a balanced BST that is simpler to maintain on deletion?

Interactive Reference: AVL Tree Visualization -- W3Schools

Part 6: Red-Black Trees -- A Different Philosophy

Why Another Balanced BST?

AVL trees enforce **strict** balance -- the height difference at every node is at most 1. This gives excellent search performance but means **more rotations** during insertions and especially deletions.

Red-Black trees take a different approach: They allow the tree to be **slightly less balanced** in exchange for **cheaper maintenance**.

The trade-off:

```
AVL:        Strictly balanced       -->  Faster search, more rotations
Red-Black:  More relaxed balance    -->  Slightly slower search, fewer rotations
```

Red-Black trees are the industry standard:

- Java: `TreeMap`, `TreeSet`
- C++ STL: `std::map`, `std::set`, `std::multimap`
- Linux kernel: Completely Fair Scheduler (CFS)
- Most language runtime libraries choose Red-Black over AVL

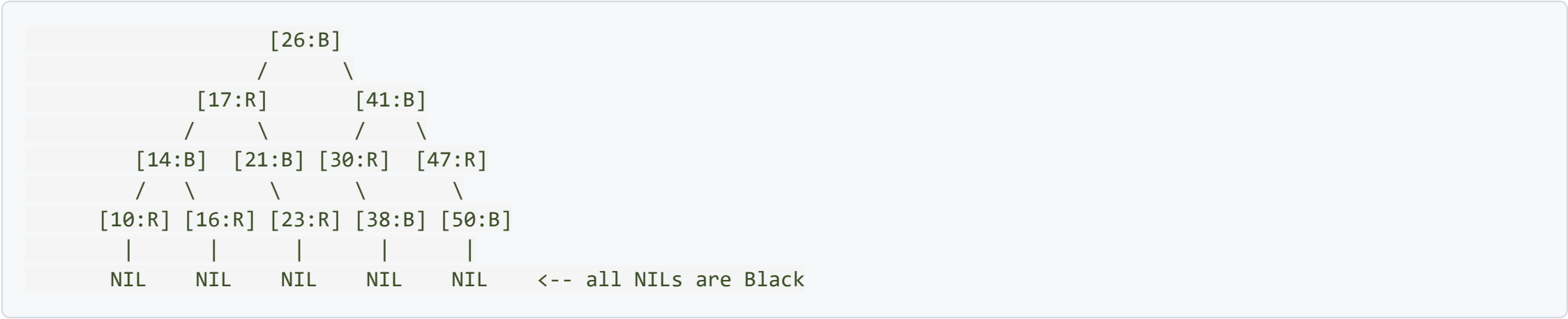
Interactive Reference: Red-Black Tree Visualization -- USF

The Five Properties of Red-Black Trees

A Red-Black tree is a BST where every node is colored either **Red** or **Black**, and the following five properties are maintained:

1. Every node is either **Red** or **Black**
2. The **root** is always **Black**
3. Every **leaf (NIL sentinel)** is **Black**
   (NIL nodes are special "empty" nodes at the bottom -- not the same as regular leaves)
4. If a node is **Red**, both its children must be **Black**
   (No two consecutive red nodes on any path)
5. Every path from any node to any of its descendant NIL nodes passes through the **same number of Black nodes** (this count is called the **Black-Height**)

Red-Black Tree: Example



Verify the properties:

- Property 1: Every node is Red or Black -- yes
- Property 2: Root (26) is Black -- yes
- Property 3: All NIL leaves are Black -- yes
- Property 4: Red 17's children are Black 14 and Black 21 -- yes (check all red nodes)
- Property 5: Count black nodes on any root-to-NIL path -- they are all equal

The Consequence: Height Guarantee

Property 5 (equal black-height on all paths) combined with **Property 4** (no consecutive reds) gives us a powerful guarantee:

The **longest path** from root to any NIL can have at most **twice** as many nodes as the **shortest path**.

Why? The shortest path is all black nodes. The longest path alternates red and black (since no two reds can be consecutive). So the longest is at most 2x the shortest.

Height bound: $h \leq 2 \times \log_2(n+1)$

Nodes	AVL Max Height	Red-Black Max Height
7	3	5
100	9	13
1,000	14	19
1,000,000	28	39

Red-Black trees are slightly taller, but the difference is a constant factor -- both are O(log n).

Red-Black Tree Operations -- Conceptual Overview

Insertion

1. Insert the new node as in a regular BST
2. Color the new node **Red** (this preserves Property 5 -- black-height unchanged)
3. **Fix violations:** The new red node might create two consecutive reds (violates Property 4)
4. Fix using **recoloring** and at most **2 rotations**

Deletion

1. Delete as in a regular BST
2. If the removed node was **Black**, the black-height is disrupted
3. Fix using **recoloring** and at most **3 rotations**

Full implementation of Red-Black insert/delete is beyond our scope. The key takeaway is the *property-based* approach: instead of tracking exact heights, we track colors and ensure the five properties hold after each operation.

AVL vs Red-Black -- Side-by-Side

Feature	AVL Tree	Red-Black Tree
Balance strictness	Strict (BF in {-1,0,+1})	Relaxed (color rules)
Maximum height	1.44 log n	2 log n
Search performance	Faster (shorter tree)	Slightly slower
Insert rotations	At most 1	At most 2 + recoloring
Delete rotations	Up to O(log n)	At most 3 + recoloring
Extra storage per node	1 integer (height)	1 bit (color)
Implementation	Moderate	More complex
Best for	Search-heavy workloads	Insert/delete-heavy workloads
Used in	Academic, GIS, compilers	Java, C++ STL, Linux kernel

Summary: AVL = better search, more maintenance. Red-Black = worse search, cheaper maintenance. Both guarantee O(log n) for all operations.

The "When Do I Use What?" Guide

Scenario	Best Choice	Why
In-memory dictionary (mostly lookups)	AVL	Strictly balanced, fastest search
Language standard library (general use)	Red-Black	Fewer rotations on insert/delete
Database index on disk	B-Tree	Minimizes disk accesses (next unit)
Priority queue / sorting	Binary Heap	O(1) find-max, O(log n) insert
Frequent search + frequent insert/delete	Red-Black	Bounded rotations on all ops
Compiler symbol table	AVL	Mostly lookups after initial build

The right data structure depends on the **workload pattern** -- not just the operation complexity.

Key Takeaways

What We Covered Today

1. AVL Trees -- Rotations and Insertion

- Balance Factor = height(left) - height(right), must be in {-1, 0, +1}
- Four cases: LL, RR (single rotation), LR, RL (double rotation)
- Insertion needs at most one rotation

2. AVL Deletion

- Same four rotation cases, but identified by child's balance factor
- Deletion can require **cascading rotations** up to O(log n)
- This is the key disadvantage of AVL trees for write-heavy workloads

3. Red-Black Trees

- Five color-based properties that guarantee O(log n) height
- More relaxed balance: height up to 2 log n (vs 1.44 log n for AVL)
- Bounded rotations: at most 2 on insert, at most 3 on delete
- Industry standard: Java, C++ STL, Linux kernel

4. AVL vs Red-Black: Search-heavy = AVL, Write-heavy = Red-Black

Review Questions

1. Insert the values **40, 20, 60, 10, 30, 50, 70, 5** into an empty AVL tree. Show the tree after each insertion and identify any rotations performed.
2. Consider inserting **1, 2, 3, 4, 5, 6, 7** into an empty AVL tree. How many rotations are needed? What does the final tree look like?
3. Why does an LR case require **two** rotations instead of one? Draw an example where a single right rotation fails to fix the imbalance.
4. Delete node **30** from the AVL tree built in question 1. Does the deletion cause any rotations? If so, which case?
5. Explain why AVL deletion may require **cascading rotations** while AVL insertion does not.
6. List the five properties of a Red-Black tree. Why does Property 4 (no consecutive reds) combined with Property 5 (equal black-height) guarantee that the longest path is at most twice the shortest?
7. A system needs to maintain a sorted collection of 10 million records, with 90% lookups and 10% inserts. Would you choose an AVL tree or a Red-Black tree? Justify your answer.
8. Why do most standard libraries (Java, C++) use Red-Black trees instead of AVL trees?

Next Lecture

Multi-Way Search Trees and B-Trees

- Stepping beyond binary -- why two children is not enough for disks
- B-Tree properties: wide, short, and perfectly balanced
- Search and insertion in B-Trees with splitting
- How databases use B-Trees to serve millions of queries